Muhammad Sohail

https://msohail.work

Firenze, Italy

✉ msohail.dev@gmail.com +39 392 803 6934

msohailse sohail0992 sohail

SUMMARY

Full-stack Software Engineer with 7+ years of experience building scalable, cloud-native applications using Node.js, TypeScript, React/Angular, and SQL/NoSQL databases. Strong background in designing microservices, APIs, and performance-optimized systems deployed on AWS and Kubernetes, with hands-on experience in CI/CD, testing, and cross-functional collaboration. AI enthusiast exploring agent-based systems and Retrieval-Augmented Generation (RAG) to build practical, data-driven automation solutions, with a strong interest in impact-driven and sustainability-focused products.

EDUCATION

UiT Norges arktiske universitet Sep 2026 – Present
Master's Degree (In Progress)

- **Relevant Coursework:** Machine Learning, Advanced Computer Security

Università degli Studi di Firenze Jul 2025 – Sep 2026
MS, Software, Science and Technology

- **Relevant Coursework:** Architecture, Model and Analysis of Cyber-Physical Systems, Data Collection and Machine Learning for Critical Cyber-Physical Systems, Distributed Programming for Web, IoT and Mobile Systems, Natural Language Processing, Statistical Methods for Computer Science

Riphah International University Spring 2018
BS, Software Engineering (CGPA 3.05)

Certifications

AWS Certified Cloud Practitioner	AWS
MongoDB Certified	MongoDB University
React Certification	TestDome
Dart and Flutter: The Complete Developer's Guide	Udemy
English Proficiency Test	Duolingo

AI & AUTOMATION

ApplyBot: Pay-per-Search AI Job-Matching Product

- Built, deployed and operate solo a production pay-per-search job-matching product live at applybot.eu (Next.js 15, Postgres on AWS RDS, Drizzle ORM, Better Auth, AWS Amplify): a customer uploads a CV and sets target roles and places, a search finds jobs, an LLM scores each against the CV, and only strong matches are returned; the customer applies themselves.
- Isolated every paid search in its own AWS Fargate container started through a launcher Lambda, with only a run-scoped token and no database access, a one-search-at-a-time limit per customer and a global concurrency cap; scoring uses a Bedrock-hosted LLM with a per-search budget and token cap.

- Implemented a central AI cost gate that enforces caps on every AI call (chat, CV fill, skills report, job runs), and a guided in-app chat assistant with strict scope, no tools or database access, cost caps per customer and per IP, and hand-off to support tickets.
- Encoded the money-back rule (refund when fewer than 20 jobs score 7 or more) in a single source file shared by the UI, the terms and the refund API; tested with Vitest and Testcontainers integration tests, with GitHub Actions CI, auto-deploy on merge and a 25-check post-release script run against the live site.

Multi-App Billing & Payments Platform with MCP Admin Server

- Built and run in production a reusable billing module (private npm package) and hosted service (TypeScript, AWS Lambda, Postgres with Drizzle, Stripe) that is the single payment system behind live apps such as ApplyBot, called over HTTP: hosted checkout, invoices, subscriptions, one-off purchases, refunds and a token wallet. Multi-merchant by design, with a separate API key, Stripe account and webhook secret per app, and an app without Stripe keys fails closed.
- Built an invoice and subscription ledger with an append-only events table: settling an invoice, extending access and writing the ledger entry happen in one database transaction so nothing applies twice. Invoices carry receipt numbers, line items, VAT/tax splits, per-currency bank details and a pay link.
- Automated settlement for card payments (Stripe, signed webhooks) and bank transfers (Wise statement matching that never guesses: a payment must name the invoice and cover at least 95% of it, and one payment settles one invoice).
- Added discount codes (percent or amount, 100% off, single-use and batches, scoped to products), an admin dashboard (password and TOTP login, CSRF protection, audit log) and a support module with secure attachments, hashed customer reply links and email alerts.
- Built an MCP server (streamable HTTP) as the owner's chat-side admin connector onto the service: payment and transaction lookup, discount-code management and a support inbox, with personal, revocable, expiring hashed keys, per-key rate limits and audit logging. It deliberately has no refund or tax tools, and customer messages are treated as untrusted text to resist prompt injection.
- Deployed on AWS Lambda behind a Cloudflare Worker routing separate hostnames; about 420 tests run against a throwaway Docker Postgres, with a weekly SonarCloud scan.

AI WhatsApp Booking Assistant for Barbershops (Multi-Tenant SaaS)

- Built a multi-tenant SaaS where each shop connects its own WhatsApp number and an LLM agent (LangChain.js, DeepSeek V3.2 on Amazon Bedrock, RAG on pgvector) handles customer chats end to end: checks availability, books, reschedules and cancels, runs a waitlist, refills cancelled slots and sends birthday offers.
- Designed an event-driven AWS pipeline: a WhatsApp bridge (whatsapp-web.js and Chromium in Docker on EC2, one session per shop) feeds a Nuxt API, then SQS with dead-letter queues, then a Lambda worker, with replies returning through a second queue and hourly jobs on EventBridge.
- Created an "Observer Harness", a separate Python service that supervises the agent in real time: Postgres triggers wake it on every customer message and agent error, and it escalates in tiers (check the message, steer the agent with a hidden note, take over with a fallback reply, then draft a fix PR for human review and alert the team). It fails open, uses a minimal-privilege database role and never messages stale conversations.
- Added agent guardrails (tool-call round limits, per-customer and per-shop daily reply caps, input size limits, per-plan usage limits, allow-listed draft-only fix PRs that are never auto-merged) and built the owner dashboard in Nuxt 4 (calendar, conversation threads, analytics, four languages) plus a super-admin console.

- Tested with 200+ automated tests using scripted fake LLMs so CI needs no cloud credentials or LLM spend, with GitHub Actions against a Postgres service container, SonarCloud and additive-only schema migrations.

Multi-Agent Engineering Team: Hermes Agents with Obsidian Memory

- Run a personal multi-agent engineering pipeline in a shared Buzz channel (Nostr-style relay), with each agent keeping its memory in an Obsidian vault: a CTO agent reads the channel, memory, vault and GitHub, delegates tickets to a developer agent and chases blockers. ApplyBot's failure watcher posts plain-language alerts to the channel and tags the CTO agent, which triages each one and hands it to a developer agent.
- The developer agent reads the repo's docs, works test-first, opens PRs and fixes review and CI comments; a PR-review agent requires green build, tests and Sonar plus a deep quality, security and architecture review, and merges only after QA sign-off; a QA agent runs the app for visual and functional checks with read-only database access, sends bugs back with repro steps and runs a production check after each merge.
- Added a reverse-engineering agent that analyzes shipped artifacts (binaries, bundles, browser surfaces) statically and at runtime, and returns cited evidence, an architecture map and a developer TODO list.

AI Agent & MCP Server: Natural-Language Invoicing Platform

- Designed and shipped a production Model Context Protocol (MCP) server (TypeScript, Express, stateless Streamable HTTP) that lets an LLM chat client operate a business invoicing platform in natural language: creating clients and invoices from a single freeform message, attaching payment links, emailing invoices and reconciling payments through tools proxied to a REST API over an internal network.
- Built an LLM drafting pipeline that turns freeform messages into structured client and invoice drafts using JSON mode with Zod schema validation (line items, currency inferred from client country, natural-language bill dates normalized to ISO), while deterministic server-side rules handle fee gross-up, currency and receiving-account selection so the model never decides monetary amounts.
- Implemented a security-first agent design: SHA-256-hashed, rotatable, brand-scoped per-admin API keys issued only by super-admins and sent via `Authorization`/`X-API-Key` headers so secrets never enter the chat transcript, owner-scoped authorization on every tool, per-IP rate limiting, and a create → human-review PDF → send workflow so the agent never emails clients without confirmation.
- Built a public AI receptionist chat strictly isolated from the agent: text-only (no tool calling), prompt-injection hardening (role sanitization, length caps, link allow-listing), rate limiting, multi-provider LLM routing with automatic fallback, and an automated boundary test that fails the build if the public chat ever imports admin code or gains tools.
- Automated payment reconciliation with an idempotent scheduled job that matches a multi-currency payment provider's read-only statement API against unpaid invoices by reference and amount, supporting USD/EUR/GBP invoices with bank/SEPA details, QR pay links and PDF generation.
- Performed a security audit and hardening of the Next.js 16 (App Router) application: patched a critical framework RCE advisory and all dependency CVEs (clean npm audit), added login brute-force lockout, password-bound session revocation and security headers (HSTS), and shipped Dockerized CI/CD to AWS via GitHub Actions with Vitest test gates.

SigenStor AI Agent: RAG-Driven Energy Consultant

- Engineered an AI agent using n8n and PostgreSQL to maintain session-aware state, leveraging historical dialogue to provide contextually accurate responses throughout the customer journey.

- Developed a RAG (Retrieval-Augmented Generation) pipeline using vector embeddings to ingest technical manuals and pricing data, enabling the LLM to deliver precise, zero-hallucination technical support.
- Integrated HomeWizard energy telemetry to automate residential system sizing, dynamically calculating battery capacity requirements based on 3-phase grid data and real-time load profiles.
- Programmed automated validation logic for installation constraints, including electrical cable cross-section requirements ($5 \times 4\text{mm}^2$) and modular stack floor-loading limits (70kg per unit).

Document Intelligence & Automated Form Processing

- Engineered an automated OCR pipeline using Google Cloud Vision, backed by an AWS Pub/Sub-based event pipeline, to digitize over 50k handwritten pamphlets from physical street campaigns, with throughput of up to 50,000 images per day, significantly reducing manual labor for administrative workflows.
- Integrated OpenAI models to perform entity extraction and KPI derivation from unstructured OCR text, enabling automated pre-filling of digital records.
- Created a dual-view interface where the system automatically pre-fills the digital form from the pamphlet image on one side and displays the actual image on the other, allowing a single operator to simply confirm the data and hit submit.

Voice Intelligence & Real-Time Transcription

- Developed a microservice leveraging WebSockets to stream call recordings to Watson Speech-to-Text, enabling real-time transcription and automated compliance monitoring.
- Architected a security layer to identify sensitive information, such as unintended requests for passcodes or restricted data, within live audio streams.
- Optimized streaming performance for high-volume call environments by integrating REST API hooks with asynchronous WebSocket events.

WORK EXPERIENCE

Trident Marketing Sep 2018 – Present

Tech Lead / Software Engineering Manager

- Led a cross-functional engineering team delivering Angular and Node.js applications, coordinating backend, frontend, and QA development across multiple production systems.
- Architected scalable microservices and hybrid mobile solutions, leveraging Node.js, TypeScript, and MongoDB to support enterprise-level requirements. Designed system architecture patterns, API gateways, and authentication flows (JWT/OAuth) for secure, cloud-native applications.
- Integrated mobile apps with external booking systems and third-party services to streamline business operations (REST, WebSockets, ElasticSearch).
- Designed and co-architected the TRA API to manage real-time data flow between multiple apps and external booking systems, leveraging REST API principles for scalability and efficiency (Node.js, TypeScript).
- Designed the TRADB (MongoDB) schema for large-scale data handling and integrations.
- Optimized system performance and scalability by managing real-time synchronization across multiple Elastic indices, applying caching strategies, and tuning database queries.
- Enabled booking-to-sales conversion by updating household records, converting leads to members, and indexing data in the Manifest System.

- Built the Engage App using Angular and TypeScript, enabling agents to track customer data (e.g., demographics, call history) and manage bookings within the same platform, significantly streamlining campaign workflows and improving agent productivity.
- Engineered the Manifest System (Angular, Node.js) to track bookings, agent performance, commissions, and the overall sales lifecycle.
- Data Visualization: Highcharts, Chart.js, Recharts, dashboard analytics, interactive reporting, real-time data visualization.
- Developed the DNQ System, a multi-part survey platform:
  - Backend uses a tree data structure for complex survey logic.
  - Frontend built with Ionic (Angular, Node.js).
- Contributed to the Lead Input Application (LIA) UI for uploading leads to S3 and splitting large PDFs (Angular, Node.js).
- Created various survey management platforms using Ionic.
- Implemented a message broker for high-volume calls, integrating with TRADB for faster processing.
- Built WebSocket-based solutions for real-time notifications, integrated with REST APIs to enhance scalability (e.g., member reservations).
- *Quality Assurance*:
  - Engineered complex Data Merging Logic for Household/Lead entities, utilizing Mocha & Chai to validate 20+ edge cases (e.g., duplicate resolution, membership inheritance).
  - Implemented a robust Test-Driven Development (TDD) workflow using JSON fixtures to simulate API responses, ensuring zero-regression deployments.
  - Developed the logic for Dynamic Meeting Payloads, validating attendee filtering rules (primary vs. secondary contacts) through automated assertion suites.
  - *CI/CD & Reporting*: Integrated the test suite into Jenkins pipelines, configuring it to capture JSON test results and generate automated build reports for immediate feedback on code stability.
- *Python*:
  - Used ChatGPT for data analysis, KPI derivation, and pre-filling forms.
  - Developed a FastAPI-based rule engine to validate data extracted via OCR against historical records to determine user eligibility and system compatibility.
  - Wrote behavior-driven tests using Behave.
  - Performed data cleaning with Pandas and built Jupyter notebooks for analysis.
- *Infrastructure & CI/CD*:
  - Optimized MongoDB cluster configurations, saving approximately $12,000 annually while maintaining system performance and reliability.
  - Led Kubernetes-based deployments and CI/CD pipeline improvements (Jenkins, GitHub Actions), incorporating Infrastructure as Code (IaC) practices to enhance scalability, resilience, and operational efficiency.
  - Run 30+ Node.js and Python microservices in production on Azure Kubernetes Service (AKS), with secrets managed in Azure Key Vault, Elastic APM monitoring across services, and Jaeger distributed tracing on high-volume consumers.
  - Maintained real-time sync with Elastic indices.
  - Created unit/behavior tests (Selenium, Chai, Mocha, Cypress) and integrated GitHub pipelines (JavaScript, Node.js).

– Configured Jenkins pipelines for automated test reporting.

Iterca Platform Mar 2026 – Sep 2026
Senior Software Engineer / Platform Engineer

- **Cloud Infrastructure & CI/CD Automation:** Engineered a scalable deployment pipeline on AWS Amplify for a distributed Next.js architecture. Resolved complex serverless injection issues, ensuring secure delivery of runtime configurations to Node.js Lambdas.
- **Complex Business Logic & Ledgers:** Designed a double-entry financial and meeting credit ledger. Implemented strict database-level concurrency controls (locks) to successfully prevent race conditions and credit exhaustion exploits during checkout.
- **Modern Web Architecture (SSR/RSC):** Architected the platform using TypeScript, React, and Next.js App Router. Bridged React Server Components (SSR) with Client Components to optimize web performance, reduce payloads, and deliver responsive UI flows.
- **Database Management & Caching:** Provisioned AWS RDS PostgreSQL within a secure VPC. Balanced database load by implementing advanced Next.js Edge caching with custom invalidation rules.
- **Platform UX & Reusable Tooling:** Elevated usability by refactoring legacy forms into state-driven React components with comprehensive error boundaries to improve developer experience and user conversion flows.
- **AI & Workflow Automation:** Engineered a standalone MCP (Model Context Protocol) Server for secure AI-driven note management, and built an autonomous webhook state machine to automate meeting tracking and financial escrow payouts for 50+ coaches.

iRide PK Jan 2025 – Sep 2025
Flutter Developer (Contract/Part-Time)

- Built the iRide ride-hailing app in Flutter, reaching 60,000+ users in Lahore, Pakistan within 2 months of launch.
- Used BLoC and Dart null safety to deliver robust, testable state management.
- Reduced MongoDB hosting costs from $1,500/mo to $300/mo (80% reduction) by implementing Redis caching, optimizing indices, and tuning queries.

Code For Pakistan (Volunteer) Mar 2021 – Present
Web Engineer/Flutter Developer

- Integrated a WhatsApp chatbot for the COVID-19 Telehealth Initiative, enabling identification and random assignment of high-risk patients to volunteer doctors (React, Node.js, PostgreSQL).
- Built a Flutter-based mobile app for lab evaluations (LAARC-standard surveys) paired with a Vue.js dashboard for performance reporting (Flutter, Vue.js, Node.js).
- Collaborated with WHO and NIH as a Solutions Engineer to architect and implement secure healthcare applications (HCAI app) using Flutter, Vue.js, Node.js, and MongoDB. Applied system design principles, microservices architecture, and performance tuning while documenting architecture and workflows to improve healthcare protocols and post-surgery infection tracking.
- Education Mentor, KP Women Program / WomenInTechPK *(2023 – Present)*: conducted training sessions (e.g., Git and version control) for participants to enhance their collaborative development workflows.
- Facilitated the interview process in 2025 to hire mentors and candidates for the KP Women Program.
- Served as speaker and mentor for the KP Government Innovation Fellowship.

Jeddah Soft Sep 2018 – Dec 2018
Mern Stack Developer (Remote Part Time)

- *Doctor App* (MySQL, React, Node.js):
  - Built a React + Bootstrap dashboard for managing doctor appointments.
  - Implemented Highcharts for admin reporting.
  - Maintained a session-based shopping cart tied to user authentication.
- *ShopAPI (Open Source):*
  - See the ShopAPI GitHub repo (Express/Node.js + WebSockets).
  - Uploaded and indexed files/images via Cloudinary.
  - Maintained session-based shopping cart logic.

xFlow Research Inc. Oct 2016 – Feb 2017
Internee

- Developed a VR basketball game for HTC Vive.

Skills

- **Languages**: TypeScript, Java, Python, Go, Node.js, Dart, C++
- **Frameworks**: Angular, React, Vue, Ionic, Express, FastAPI, Flutter, Quarkus, Pandas, NumPy, Behave, Mocha, Chai, Cypress, Selenium, Protractor, Highcharts, Recharts, Bootstrap
- **Databases**: MongoDB, MySQL, MsSQL, PostgreSQL, ElasticSearch
- **AI & Automation**: n8n, RAG, OpenAI, IBM Watson (STT), Google Cloud Vision, GraphQL
- **Cloud & DevOps**: AWS (SNS/SQS), Azure (AKS, Key Vault), GCP (Compute Engine, Networking), Docker, Kubernetes, Helm, Istio, CI/CD, Jenkins, GitHub Actions, Apache, RabbitMQ, Kafka, Message Brokers, WebSockets, Cloudinary, Git (GitHub, GitLab)
- **Observability & Performance**: Prometheus, Grafana, Jaeger, Elastic APM, PromQL, Distributed Tracing, Load Testing (Locust), Capacity Planning, Queueing Models
- **Architecture & Practices**: System Design, Microservices, API Gateway, Event-Driven Systems, Performance Optimization, Scalability, Cloud Native Architecture, System Architecture Patterns, Hybrid Mobile Architecture, Distributed Systems Architecture
- **Tools**: Jupyter, NetBeans, Eclipse, VS Code, HTC Vive

Projects & Research

Microservices Performance Engineering: Observability, Capacity Modelling & Scaling
Kubernetes, Helm, Istio (Envoy), Prometheus, Grafana, Jaeger, PromQL, Locust, JMT

- Deployed a polyglot gRPC microservices system (Go, Java, .NET, Node.js, Python) on Kubernetes with an Istio service mesh, and provisioned the observability stack (Prometheus, Grafana, Jaeger) via Helm (kube-prometheus-stack).
- Set up zero-code distributed tracing through Envoy sidecars (Zipkin to Jaeger, 100% sampling) and wrote PromQL queries for per-service throughput, mean latency and p95; automated collection of metrics and service dependency graphs into a single script.
- Diagnosed a CrashLoopBackOff caused by CPU throttling that failed gRPC liveness probes on .NET and Node.js services, and retuned container CPU/memory limits to stabilise the cluster.
- Load-tested with Locust from 10 to 200 concurrent users and built a queueing-network capacity model (validated in JMT) that predicted response times within 10% up to 89% bottleneck utilisation and located the frontend's saturation point (∼58 req/s).

- Scaled the bottleneck service horizontally based on the model, cutting mean response time by 79% (199 to 41 ms) and p95 latency by 82% (807 to 144 ms) at 100 concurrent users, with a cost analysis of the added CPU/memory.

Honeypot Analysis and Visualization Tool (FYP)
Data Analysis, MongoDB, Geo-Mapping

- Analyzed security data from various honeypots (Dionaea, Suricata, Kippo).
- Visualized attack data intensity on interactive geographical maps.
- Centralized data ingestion pipeline in MongoDB for increased query performance.

SWAM: Scalable Event-Driven Incident Management System
Quarkus (Jakarta EE, JAX-RS), Angular, PostgreSQL, Kafka, Kubernetes

- Designed a hexagonal (ports-and-adapters) architecture isolating the core domain behind a REST API built with Jakarta EE on Quarkus, using a lightweight CQRS read path.
- Implemented event-driven duplicate detection: the API publishes incident-creation events to a Kafka topic, consumed asynchronously by an independent analyzer service using fuzzy text-similarity matching, reporting back without blocking the write path.
- Enforced role-based access control with incident visibility scoped to departments and transactional write operations.
- Deployed via Docker Compose locally and Kubernetes in production, with the API service horizontally autoscaled.
- Verified the event-driven flow with Testcontainers-based integration tests.

Encrypted Document Vault with Sharing Control
Go (Golang), Angular, RabbitMQ, MongoDB, Docker, WebSockets

- Engineered a secure, distributed document vault platform for sharing sensitive files with full owner control.
- Designed a multi-instance distributed architecture to support horizontal scaling, ensuring system-wide state consistency by using RabbitMQ (AMQP) for reliable event dispatching across instances.
- Implemented a real-time "Eye Tracking System" via WebSockets to instantly notify document owners of current viewers, tracking access by IP address and browser fingerprint.
- Enforced comprehensive security with AES-256 file encryption, configurable burn-once links, and instant system-wide access revocation.

Achievements & Leadership

Università degli Studi di Firenze Jul 2025 – Sep 2026
Student Representative

- Member of the Commissione Paritetica Docenti-Studenti CdS, serving as a bridge between the student body and faculty.
- Contribute to the evaluation and improvement of teaching quality and academic services within the Software, Science and Technology program.

WomenInTechPK Feb 2023 – Present
Education Mentor

- Lead hands-on sessions and mentoring programs to help participants gain real-world experience with modern tech tools, frameworks, and workflows used in industry and civic tech.

TEN 2017 – 2018

Alumni Relations Manager

- Managed alumni network relations and engagement.